

UNIVERSITÀ DELLA CALABRIA

DIPARTIMENTO DI INGEGNERIA INFORMATICA,  
MODELLISTICA,  
ELETTRONICA E SISTEMISTICA (DIMES)

Corso di Laurea Magistrale in Ingegneria Informatica(Cybersecurity)  
Corso di Sistemi Distribuiti e Cloud Computing — A.A. 2025/2026

---

# DocuMatch

## Relazione di Progetto

*Sistema Cloud-Native ed Intelligente per la Gestione,  
Ricerca Semantica Vettoriale ed Estrazione OCR  
di schede Documentali Comunali tramite AI Generativa*

---

*Candidato:*

**Manuel Amodio**  
Matricola: **276571**

*Docenti:*

**Prof. Domenico Talia**  
**Prof. Loris Belcastro**

Piattaforma Cloud: **Microsoft Azure**

Anno Accademico 2025/2026

# Indice

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione e Requisiti del Progetto</b>                        | <b>4</b>  |
| 1.1      | Contesto e Motivazione . . . . .                                    | 4         |
| 1.2      | Requisiti Funzionali . . . . .                                      | 4         |
| 1.3      | Requisiti Non Funzionali . . . . .                                  | 5         |
| <b>2</b> | <b>Architettura del Sistema</b>                                     | <b>6</b>  |
| 2.1      | Panoramica Generale . . . . .                                       | 6         |
| 2.2      | Schema Architetturale . . . . .                                     | 7         |
| 2.3      | Stack Tecnologico . . . . .                                         | 8         |
| 2.4      | Alternative Valutate e Scartate . . . . .                           | 9         |
| <b>3</b> | <b>Layer di Persistenza e Modello dei Dati</b>                      | <b>10</b> |
| 3.1      | Architettura Dual-Mode del Database . . . . .                       | 10        |
| 3.2      | Modelli SQLAlchemy . . . . .                                        | 10        |
| 3.2.1    | Modello <b>Document</b> . . . . .                                   | 10        |
| 3.2.2    | Modello <b>User</b> . . . . .                                       | 10        |
| 3.2.3    | Pannello di Amministrazione e Configurazione . . . . .              | 10        |
| <b>4</b> | <b>Servizi Cloud Microsoft Azure</b>                                | <b>12</b> |
| 4.1      | Pattern Dual-Mode e Dependency Inversion . . . . .                  | 12        |
| 4.2      | Azure AI Document Intelligence (OCR) . . . . .                      | 12        |
| 4.3      | Azure Blob Storage . . . . .                                        | 12        |
| 4.4      | Azure OpenAI Service (Chatbot RAG ed Estrazione Metadati) . . . . . | 13        |
| 4.4.1    | Architettura del Chatbot RAG . . . . .                              | 13        |
| 4.4.2    | Estrazione Metadati OCR tramite LLM . . . . .                       | 13        |
| 4.5      | Servizio Email SMTP (Gmail) . . . . .                               | 14        |
| 4.6      | Riepilogo Servizi Cloud . . . . .                                   | 14        |
| <b>5</b> | <b>Pipeline di Intelligenza Artificiale</b>                         | <b>15</b> |
| 5.1      | Named Entity Recognition con spaCy . . . . .                        | 15        |
| 5.2      | Generazione Embeddings con SentenceTransformers . . . . .           | 15        |
| 5.3      | Cosine Similarity . . . . .                                         | 15        |
| 5.4      | Ricerca Ibrida: Cosine + Keyword Boosting . . . . .                 | 16        |
| <b>6</b> | <b>API REST — Backend FastAPI</b>                                   | <b>17</b> |
| 6.1      | Struttura del Progetto Backend . . . . .                            | 17        |
| 6.2      | Endpoint di Autenticazione . . . . .                                | 17        |
| 6.3      | Endpoint Documenti . . . . .                                        | 17        |

|           |                                                                       |           |
|-----------|-----------------------------------------------------------------------|-----------|
| 6.4       | Endpoint Chatbot RAG . . . . .                                        | 18        |
| 6.5       | Filtri di Ricerca e Pre-Filtering . . . . .                           | 18        |
| 6.6       | Importazione Massiva con Pandas . . . . .                             | 19        |
| <b>7</b>  | <b>Frontend — Dashboard Istituzionale React</b>                       | <b>20</b> |
| 7.1       | Architettura a Servizi e Viste . . . . .                              | 20        |
| 7.1.1     | Componenti e Viste principali . . . . .                               | 20        |
| 7.1.2     | Servizi Frontend . . . . .                                            | 20        |
| 7.2       | Design System . . . . .                                               | 21        |
| 7.3       | Funzionalità UX Avanzate . . . . .                                    | 21        |
| 7.3.1     | Filtri Multi-Selezione con Logica AND/OR . . . . .                    | 21        |
| 7.3.2     | Indicatore di Affinità Semantica . . . . .                            | 21        |
| 7.3.3     | Selezione Multipla e Bulk Export . . . . .                            | 22        |
| 7.3.4     | Sicurezza Token: sessionStorage vs localStorage . . . . .             | 22        |
| 7.4       | URL Relativi e Proxy . . . . .                                        | 22        |
| 7.5       | Autenticazione Stateless nel Frontend . . . . .                       | 22        |
| <b>8</b>  | <b>Containerizzazione e Deploy Cloud (Azure)</b>                      | <b>23</b> |
| 8.1       | Dockerfile Backend . . . . .                                          | 23        |
| 8.2       | Multi-Stage Build del Frontend . . . . .                              | 24        |
| 8.3       | Nginx con envsubst . . . . .                                          | 24        |
| 8.4       | Deploy su Azure Container Apps . . . . .                              | 25        |
| <b>9</b>  | <b>Sicurezza e Autenticazione</b>                                     | <b>26</b> |
| 9.1       | Token JWT (HS256 / HMAC-SHA256) Stateless . . . . .                   | 26        |
| 9.2       | RBAC: API Pubbliche e Protette in Scrittura . . . . .                 | 26        |
| 9.3       | Password Hashing con Bcrypt . . . . .                                 | 26        |
| 9.4       | Reset Password con Token Monouso . . . . .                            | 26        |
| 9.5       | Protezione Anti-ZipBomb . . . . .                                     | 27        |
| 9.6       | Sistema di Accountability (Audit Log) . . . . .                       | 27        |
| <b>10</b> | <b>Sistema di Notifiche Schedulate</b>                                | <b>28</b> |
| 10.1      | APScheduler in Background . . . . .                                   | 28        |
| <b>11</b> | <b>Integrazione dell'IA Generativa — Dichiarazione di Trasparenza</b> | <b>29</b> |
| 11.1      | Strumento Utilizzato . . . . .                                        | 29        |
| 11.2      | Distribuzione del Lavoro . . . . .                                    | 29        |
| 11.3      | Prompt Principali Utilizzati . . . . .                                | 30        |
| 11.3.1    | Fase 1: UI/UX e Frontend React . . . . .                              | 30        |
| 11.3.2    | Fase 2: Backend e Integrazione API . . . . .                          | 31        |
| 11.3.3    | Fase 3: Intelligenza Artificiale e Cloud . . . . .                    | 31        |
| 11.3.4    | Fase 4: DevOps, Debugging e Consegna . . . . .                        | 32        |
| 11.3.5    | Fase 5: UX Avanzata e Correzioni Frontend . . . . .                   | 32        |
| 11.3.6    | Fase 6: Deploy Cloud e Ottimizzazioni Azure . . . . .                 | 34        |

|                                         |           |
|-----------------------------------------|-----------|
| <b>12 Conclusioni e Sviluppi Futuri</b> | <b>36</b> |
| 12.1 Risultati Raggiunti . . . . .      | 36        |
| 12.2 Competenze Acquisite . . . . .     | 36        |
| 12.3 Sviluppi Futuri . . . . .          | 36        |

# Capitolo 1

## Introduzione e Requisiti del Progetto

### 1.1 Contesto e Motivazione

La digitalizzazione dei documenti amministrativi comunali è una delle sfide più concrete e diffuse nella Pubblica Amministrazione italiana. I documenti amministrativi, restano spesso disponibili soltanto come scansioni non ricercabili o come fascicoli cartacei. Questo rende estremamente onerosa la consultazione per i funzionari e quasi impossibile per i comuni cittadini.

Il progetto **DocuMatch** nasce per rispondere a questa esigenza reale, sfruttando le più moderne tecnologie di *Cloud Computing*, *Intelligenza Artificiale* e *Ricerca Vettoriale Semantica* per costruire un sistema documentale istituzionale completo, scalabile e accessibile.

L'applicativo supera i limiti della classica ricerca testuale (keyword-based) integrando strumenti di AI che comprendono il significato della query dell'utente indipendentemente dalle parole esatte utilizzate.

### 1.2 Requisiti Funzionali

Tutti gli obiettivi definiti dalla traccia assegnata sono stati pienamente raggiunti:

1. **Architettura Cloud-Native su Microsoft Azure:** esecuzione su servizi gestiti Azure (Container Apps, Blob Storage, Document Intelligence, OpenAI).
2. **OCR Automatico:** estrazione del testo dalle copertine tramite Azure AI Document Intelligence.
3. **Ricerca Semantica Vettoriale:** indicizzazione tramite embedding *paraphrase-multilingual-MiniLM-L12-v2* e cosine similarity.
4. **Caricamento Massivo:** supporto CSV/JSON con calcolo automatico embedding per ogni record.
5. **AI Generativa:** Azure OpenAI per spiegazioni e chatbot RAG.

6. **Notifiche Schedulate:** APScheduler invia email agli operatori per atti in scadenza a 15, 7 e 1 giorno.
7. **Autenticazione e Sicurezza:** token HMAC-SHA256, bcrypt, reset password.
8. **Containerizzazione:** Docker + Docker Compose + Azure Container Apps.
9. **Interfaccia Istituzionale:** SPA React con UI custom esportata tramite FigmaMake.

## 1.3 Requisiti Non Funzionali

- **Portabilità:** pattern *Dual-Mode* — sviluppo offline senza utilizzo di servizi cloud, deploy cloud senza modifiche al codice.
- **Scalabilità:** Azure Container Apps con auto-scaling su Kubernetes.
- **Manutenibilità:** codice SOLID (DIP), struttura a 3 strati.
- **Sicurezza:** CORS per dominio, bcrypt, token con scadenza, anti-timing.
- **Osservabilità:** healthcheck Docker, endpoint root JSON per load balancer.

# Capitolo 2

## Architettura del Sistema

### 2.1 Panoramica Generale

DocuMatch adotta un'architettura **Client-Server a tre strati** con Separation of Concerns:

1. **Presentation Layer (Frontend):** SPA React 19 + TypeScript + Vite, servita da container Nginx con reverse proxy.
2. **Business Logic Layer (Backend):** API REST FastAPI (Python 3.9), orchestratore di tutti i servizi (NLP, OCR, storage, email, AI).
3. **Data & Storage Layer:** SQLite (sviluppo) / PostgreSQL (produzione) tramite SQLAlchemy ORM; Azure Blob Storage per file binari.

## 2.2 Schema Architetture

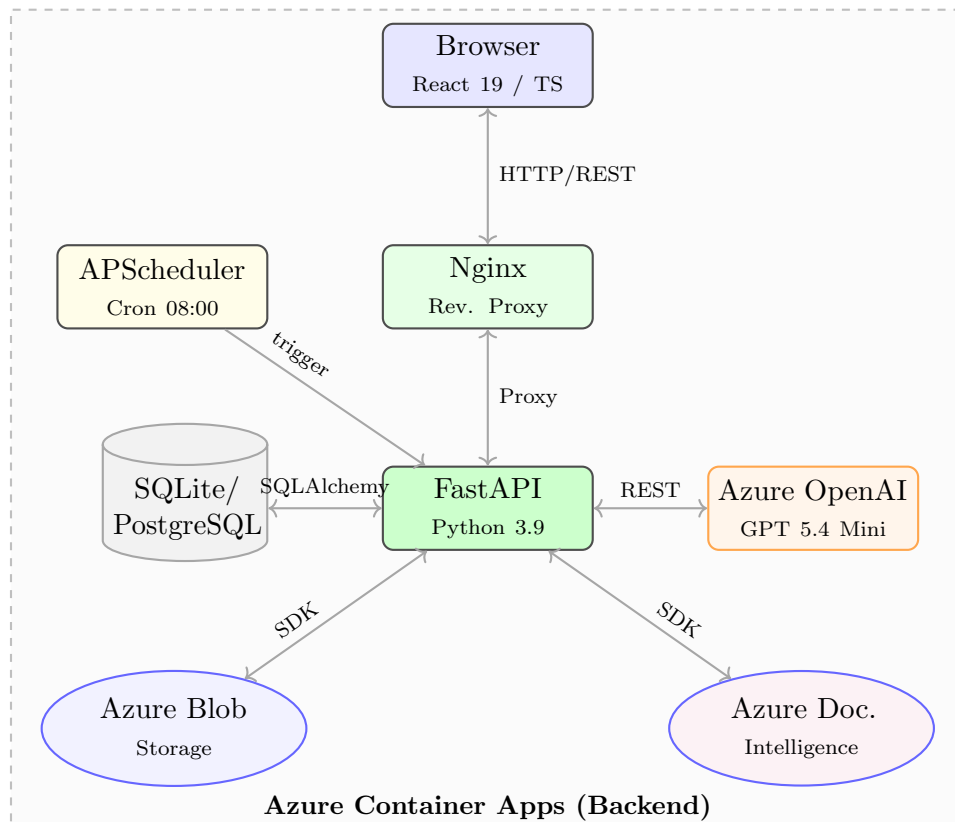


Figura 2.1: Schema architetturale di DocuMatch con tutti i servizi Azure

Il flusso di una ricerca semantica:

1. Il browser invia `GET /api/v1/documents/search?query_testo=lavori+stradali`.
2. Nginx la inoltrata al container backend FastAPI.
3. FastAPI genera l'embedding della query con SentenceTransformers.
4. Calcola la cosine similarity rispetto a tutti i documenti nel DB.
5. La risposta JSON torna al frontend con card e punteggi di rilevanza.



## 2.3 Stack Tecnologico

| Componente                 | Libreria/Versione             | Motivazione                               |
|----------------------------|-------------------------------|-------------------------------------------|
| <b>Framework API</b>       | FastAPI 0.110.0               | ASGI, Swagger auto-generato, async nativo |
| <b>Server ASGI</b>         | Uvicorn 0.28.0                | Ad alte prestazioni per FastAPI           |
| <b>ORM Database</b>        | SQLAlchemy 2.0.28             | Astrazione SQLite/PostgreSQL trasparente  |
| <b>Validazione</b>         | Pydantic 2.6.4                | Schema dati con validazione automatica    |
| <b>NLP / NER</b>           | spaCy 3.7.4                   | Riconoscimento entità in italiano         |
| <b>Embeddings</b>          | SentenceTransformers 2.5.1    | Vettori semantici multilingua a 384 dim   |
| <b>Calcolo vettoriale</b>  | NumPy 1.26.4                  | Cosine similarity e operazioni vettoriali |
| <b>Importazione dati</b>   | Pandas 2.2.1                  | Parsing CSV con separatori ambigui        |
| <b>Hashing password</b>    | Passlib[bcrypt] 1.7.4         | Key stretching resistente a GPU           |
| <b>Token auth</b>          | HMAC-SHA256 (stdlib)          | Stateless con firma e scadenza            |
| <b>Scheduler</b>           | APScheduler 3.10.4            | Cron job in background                    |
| <b>Azure OCR</b>           | azure-ai-formrecognizer 3.3.2 | Document Intelligence SDK                 |
| <b>Azure Storage</b>       | azure-storage-blob 12.19.1    | Blob Storage SDK                          |
| <b>Azure AI</b>            | openai 1.14.1                 | Client Azure OpenAI e OpenAI API          |
| <b>DB Produzione</b>       | psycopg2-binary 2.9.9         | Driver PostgreSQL                         |
| <b>Build tool frontend</b> | Vite                          | HMR istantaneo, ESBuild                   |
| <b>UI framework</b>        | React 19 + TypeScript         | Componenti dichiarativi, design to code   |

Tabella 2.1: Stack tecnologico completo di DocuMatch

## 2.4 Alternative Valutate e Scartate

**VM (IaaS) vs Container Apps (PaaS):** La VM richiede gestione del Guest OS. Container Apps (su Kubernetes gestito) astrae l'infrastruttura e offre auto-scaling, networking e TLS automatici.

**SQL LIKE vs Ricerca Semantica:** LIKE '%lavori stradali%' non trova "rifacimento manto stradale". La cosine similarity su vettori semantici cattura la similarità concettuale.

**Tesseract vs Azure AI Document Intelligence:** Tesseract ha performance inferiori su scansioni reali di bassa qualità. Il servizio cloud è ottimizzato per documenti burocratici e atti ufficiali.

**GPT 5.6 Sol vs GPT 5.4 Mini:** Inizialmente si era valutato l'utilizzo del modello **GPT 5.6 Sol** (estremamente preciso), ma i suoi tempi di risposta prolungati sono risultati incompatibili con la necessità di interazione rapida e fluida richiesta dal cliente, che non vuole attendere. Si è scelto quindi **GPT 5.4 Mini**: pur essendo intrinsecamente meno preciso, l'adozione di un *prompt engineering* aggiuntivo ha permesso di colmare il divario qualitativo, raggiungendo gli stessi risultati in maniera decisamente più veloce.

**Azure Communication Services vs Gmail SMTP:** A causa di rigide limitazioni imposte da Azure per l'abilitazione del servizio di posta elettronica sui nuovi account, si è optato per l'affidabile e immediato server SMTP di Gmail per la gestione delle email transazionali.

**SQLite in produzione vs PostgreSQL:** SQLite non è adatto al deployment multi-processo. L'astrazione SQLAlchemy rende il cambio trasparente.

# Capitolo 3

## Layer di Persistenza e Modello dei Dati

### 3.1 Architettura Dual-Mode del Database

Il backend non dipende da un database specifico ma dall'astrazione **SQLAlchemy**. La connection string viene letta a runtime da `DATABASE_URL`: In sviluppo viene usato SQLite (file locale), in produzione PostgreSQL su Azure. Il cambio è completamente trasparente per il codice applicativo.

### 3.2 Modelli SQLAlchemy

#### 3.2.1 Modello Document

Entità centrale. Le liste (uffici, firmatari, frazioni) sono serializzate come JSON string poiché SQLite non supporta array nativi:

#### 3.2.2 Modello User

Gestisce l'autenticazione degli operatori e l'eventuale recupero password. Le caratteristiche principali includono:

- **ID UUID (String 36)**: Utilizzato al posto di ID sequenziali per prevenire attacchi di tipo Insecure Direct Object Reference (IDOR).
- **Hashing bcrypt**: La password non viene mai salvata in chiaro ma protetta da algoritmi crittografici *salt-based*.
- **Flow di Reset Password**: Comprende i campi `reset_token` (monouso) e `reset_token_expiry` (validità 15 minuti) per consentire un ripristino sicuro delle credenziali in caso di smarrimento.

#### 3.2.3 Pannello di Amministrazione e Configurazione

Il pannello admin permette la gestione centralizzata delle seguenti entità e sezioni:

- **Operatori:** gestione degli utenti del sistema (creazione account, reset password, permessi).
- **Registro Attività:** monitoraggio delle operazioni (Audit Log) per garantire la tracciabilità e la sicurezza.
- **Uffici:** uffici comunali (Ufficio Tecnico, Servizi Sociali, ...)
- **Tipologie:** categorie documentali (Delibera di Giunta, Ordinanza, ...)
- **Firmatari:** funzionari abilitati alla firma degli atti.
- **Frazioni:** zone geografiche del territorio comunale.

# Capitolo 4

## Servizi Cloud Microsoft Azure

### 4.1 Pattern Dual-Mode e Dependency Inversion

Ogni servizio cloud adotta il **design pattern Strategy** con DIP (SOLID):

1. Classe astratta (contratto).
2. Implementazione cloud con API Azure reali.
3. Implementazione mock funzionale per sviluppo offline.

La selezione avviene a runtime: se le variabili d'ambiente Azure sono impostate si usa il servizio cloud; altrimenti si attiva automaticamente il mock locale.

### 4.2 Azure AI Document Intelligence (OCR)

**Motivazione:** garantisce alta accuratezza su scansioni di qualità variabile, gestisce layout complessi e tabelle degli atti ufficiali. A differenza di Tesseract, il servizio cloud è ottimizzato per documenti burocratici reali. Si usa il modello pre-addestrato `prebuilt-document`.

### 4.3 Azure Blob Storage

**Motivazione:** Object Storage per immagini delle copertine. Vantaggi rispetto al disco locale:

- **Persistenza stateless:** il container può essere riavviato/replicato senza perdere file.
- **URL pubblici permanenti:** ogni blob ha un URL diretto senza endpoint di download.
- **Ridondanza geografica (LRS/GRS):** replica automatica per alta disponibilità.

## 4.4 Azure OpenAI Service (Chatbot RAG ed Estrazione Metadati)

Integra il modello GPT 5.4 Mini per:

1. **Estrazione Metadati OCR:** trasformazione del testo OCR destrutturato in JSON strutturato.
2. **Spiegazioni semantiche:** linguaggio naturale per i risultati dei documenti correlati.
3. **Chatbot RAG:** interagisce con l'archivio testuale limitando le allucinazioni.

### 4.4.1 Architettura del Chatbot RAG

Il chatbot segue il pattern **RAG (Retrieval-Augmented Generation)**:

1. L'utente invia un messaggio, insieme all'ID del documento corrente aperto e allo **storico completo** della conversazione.
2. Il backend esegue una ricerca semantica e recupera i 3 documenti più rilevanti per la domanda posta.
3. Il documento aperto viene etichettato come [DOCUMENTO CORRENTE], con i suoi metadati espliciti (firmatari, uffici, date) e fino a 3000 caratteri di testo estratto per contestualizzare l'LLM.
4. Lo storico conversazione viene inviato come array di messaggi **role/content**, permettendo al modello di rispondere in modo contestuale.
5. L'LLM risponde in testo puro, senza Markdown (niente asterischi o elenchi puntati), basandosi esclusivamente sui documenti dell'archivio locale.

La scelta del modello GPT 5.4 Mini e il confronto con i modelli di ragionamento sono descritti nella sezione *Alternative Valutate e Scartate* (Capitolo 2).

### 4.4.2 Estrazione Metadati OCR tramite LLM

Un prompt di sistema invia il testo OCR grezzo ad Azure OpenAI e ottiene un JSON strutturato pronto per il database:

#### System Prompt — Estrazione Metadati OCR da LLM

*“Sei un assistente specializzato nell'estrazione di metadati da atti pubblici comunali italiani. Leggi il testo OCR e restituisci ESCLUSIVAMENTE un JSON valido senza Markdown con le chiavi: **nome** (tipologia + numero atto, es. Ordinanza Sindacale n. 12 del 05/06/2026), **descrizione** (max 2 frasi), **tipologia** (una tra le categorie previste), **data** (YYYY-MM-DD), **data\_scadenza** (YYYY-MM-DD o null), **uffici** (array), **firmatari** (array), **frazioni** (array o 'TUTTE').”*

Questo approccio converte il testo grezzo OCR in un record database completo in un'unica chiamata API, eliminando qualsiasi inserimento manuale.

## 4.5 Servizio Email SMTP (Gmail)

L'invio della posta è affidato al server SMTP di Gmail (le motivazioni della scelta sono descritte nel Capitolo 2). Il servizio gestisce:

1. **Reset Password:** token monouso inviato via email, valido 15 minuti.
2. **Notifiche di scadenza:** email HTML agli operatori a 15, 7 e 1 giorno.

### Modalità locale per le email

In assenza delle credenziali SMTP (Gmail), il servizio scrive il corpo dell'email in un file `.txt` nella cartella `backend/mock_emails/`.

## 4.6 Riepilogo Servizi Cloud

| Servizio Azure                   | Scopo                     | SDK Python              | Fallback           |
|----------------------------------|---------------------------|-------------------------|--------------------|
| <b>Azure AI Doc Intelligence</b> | OCR da immagini           | azure-ai-formrecognizer | MockOCRService     |
| <b>Azure Blob Storage</b>        | Archiviazione immagini    | azure-storage-blob      | LocalStorage       |
| <b>Azure OpenAI Service</b>      | Chatbot RAG + Spiegazioni | openai                  | spaCy fallback     |
| <b>Gmail SMTP</b>                | Invio Email               | smtplib                 | File .txt su disco |
| <b>Azure Container Apps</b>      | Hosting container         | Docker / az CLI         | docker compose     |
| <b>Azure Container Registry</b>  | Registro immagini         | docker push             | Registry locale    |
| <b>Azure DB PostgreSQL</b>       | DB in produzione          | psycopg2-binary         | SQLite locale      |

Tabella 4.1: Tutti i servizi Azure utilizzati da DocuMatch

## Capitolo 5

# Pipeline di Intelligenza Artificiale

### 5.1 Named Entity Recognition con spaCy

La libreria **spaCy** con il modello `it_core_news_sm` identifica automaticamente dal testo OCR:

- **PER** (Person): firmatari dell'atto.
- **ORG** (Organization): uffici coinvolti.
- **LOC** (Location): frazioni o zone geografiche.

### 5.2 Generazione Embeddings con SentenceTransformers

Il modello `paraphrase-multilingual-MiniLM-L12-v2` (50+ lingue, incluso l'italiano) converte ogni atto in un **vettore denso a 384 dimensioni**:

L'embedding viene ricalcolato: all'inserimento, durante OCR, nel bulk import, e ad ogni modifica (PUT), mantenendo il vettore sempre sincronizzato.

### 5.3 Cosine Similarity

Metrica geometrica che misura l'angolo  $\theta$  tra i vettori di query e documento. Valori vicini a 1 indicano alta affinità semantica:

$$\text{sim}(A, B) = \cos \theta = \frac{A \cdot B}{\|A\| \cdot \|B\|} = \frac{\sum_{i=1}^{384} A_i B_i}{\sqrt{\sum_{i=1}^{384} A_i^2} \cdot \sqrt{\sum_{i=1}^{384} B_i^2}} \quad (5.1)$$



## 5.4 Ricerca Ibrida: Cosine + Keyword Boosting

Per migliorare la precisione, il sistema combina la cosine similarity con un boost morfologico sugli *stem* delle parole:

- **Estrazione e Stemming Custom:** La query dell'utente viene ripulita dalle *stopwords* burocratiche (es. "comune", "delibera", "il", "per"). Le parole rimanenti passano attraverso una funzione di *stemming* personalizzata per l'italiano (`clean_and_stem`) che rimuove suffissi verbali e plurali per ottenere la radice lessicale pura (es. "manutenzione" e "manutenzioni" → "manutenzi-").
- **Keyword Boosting (+0.05):** Se la radice estratta dalla query compare testualmente nel nome, nella descrizione o nel testo OCR del documento, il punteggio di similarità coseno base riceve un *boost* additivo (es. +0.05 per ogni occorrenza).

Questo approccio garantisce che i documenti in cui è presente esplicitamente la parola chiave cercata ricevano una penalizzazione minore rispetto ai documenti semanticamente affini ma che non contengono il termine esatto, unendo l'intelligenza contestuale dei transformer con la garanzia di esattezza della ricerca testuale classica.

# Capitolo 6

## API REST — Backend FastAPI

### 6.1 Struttura del Progetto Backend

### 6.2 Endpoint di Autenticazione

| Metodo | Endpoint                              | Accesso       | Descrizione                   |
|--------|---------------------------------------|---------------|-------------------------------|
| POST   | /api/v1/auth/login                    | Pubblico      | Login, JWT Bearer Token       |
| POST   | /api/v1/auth/forgot-password          | Pubblico      | Richiesta reset password      |
| POST   | /api/v1/auth/reset-password           | Pubblico      | Conferma reset con token      |
| GET    | /api/v1/auth/users                    | Admin<br>Sup. | Lista operatori               |
| POST   | /api/v1/auth/users                    | Admin<br>Sup. | Crea nuovo operatore          |
| DELETE | /api/v1/auth/users/{username}         | Admin<br>Sup. | Rimuove operatore             |
| POST   | /api/v1/auth/trigger-expiration-check | Auth          | Trigger manuale cron scadenze |
| GET    | /api/v1/audit/                        | Admin<br>Sup. | Registro attività (Audit Log) |

Tabella 6.1: Endpoint di autenticazione e gestione utenti

### 6.3 Endpoint Documenti

| Metodo | Endpoint           | Accesso  | Descrizione    |
|--------|--------------------|----------|----------------|
| GET    | /api/v1/documents/ | Pubblico | Lista archivio |

| Metodo | Endpoint                                 | Accesso  | Descrizione                |
|--------|------------------------------------------|----------|----------------------------|
| GET    | /api/v1/documents/search                 | Pubblico | Ricerca semantica + filtri |
| POST   | /api/v1/documents/search-by-image        | Pubblico | Ricerca per foto (OCR)     |
| GET    | /api/v1/documents/{id}                   | Pubblico | Dettaglio atto             |
| GET    | /api/v1/documents/{id}/recommendations   | Pubblico | Top 5 correlati            |
| POST   | /api/v1/documents/analyze-ocr            | Auth     | Digitalizza copertina      |
| POST   | /api/v1/documents/                       | Auth     | Inserimento manuale        |
| POST   | /api/v1/documents/import-bulk            | Auth     | Bulk ZIP (JSON/CSV)        |
| POST   | /api/v1/documents/export-bulk            | Auth     | Esporta selezione in ZIP   |
| POST   | /api/v1/documents/bulk-delete            | Auth     | Eliminazione multipla      |
| POST   | /api/v1/documents/recalculate-embeddings | Auth     | Ricalcola embedding        |
| POST   | /api/v1/documents/bulk-resolve-entities  | Auth     | Risoluzione entità batch   |
| PUT    | /api/v1/documents/{id}                   | Auth     | Modifica atto              |
| DELETE | /api/v1/documents/{id}                   | Auth     | Eliminazione               |

Tabella 6.2: Endpoint di gestione documenti

## 6.4 Endpoint Chatbot RAG

| Metodo | Endpoint      | Accesso  | Descrizione                  |
|--------|---------------|----------|------------------------------|
| POST   | /api/v1/chat/ | Pubblico | Interroga l'archivio via LLM |

Tabella 6.3: Endpoint chatbot RAG

## 6.5 Filtri di Ricerca e Pre-Filtering

Il sistema implementa un approccio di **Ricerca Ibrida** che combina la precisione dei filtri deterministici (sui metadati strutturati) con la flessibilità e la comprensione del contesto della ricerca vettoriale (sui testi destrutturati).

I filtri strutturati combinati tramite l'interfaccia utente includono:

- **Ricerca Semantica:** query testuale in linguaggio naturale elaborata dal modello SentenceTransformer.

- **Tipologia:** selezione multipla delle categorie documentali (con logica AND/OR).
- **Anno:** selezione dell'anno di pubblicazione (con logica AND/OR), in sostituzione al classico e meno intuitivo range di date.
- **Ufficio, Firmatario, Frazione:** filtri a selezione multipla per entità organizzative e geografiche (con logica AND/OR).

L'algoritmo opera in due step logici (approccio *pre-filtering*):

1. I filtri hard (esattezza) vengono applicati per primi, riducendo lo spazio di ricerca e scartando a priori i documenti che non rispettano i vincoli burocratici formali.
2. La *cosine similarity* (pertinenza) viene calcolata esclusivamente sul sottoinsieme filtrato.

Questo meccanismo ibrido abbatta drasticamente i tempi di calcolo vettoriale e unisce l'affidabilità di un database classico con l'intelligenza di un motore semantico.

#### Wildcard Frazioni

Un documento con frazioni impostato a "TUTTE" viene sempre incluso nei risultati di ricerca per zona, bypassando il filtro restrittivo.

## 6.6 Importazione Massiva con Pandas

L'endpoint `POST /api/v1/documents/import-bulk` accetta archivi ZIP contenenti immagini e un file di metadati (`dati.json` oppure `dati.csv`). Il parsing del CSV è delegato a **Pandas** che gestisce automaticamente separatori ambigui, encoding diversi e valori nulli (NaN). Ogni documento viene indexato (embedding calcolato) al momento dell'ingestione. Se il campo `testo_estratto` è vuoto o assente, il documento viene accodato per elaborazione OCR asincrona in background, senza bloccare la risposta all'amministratore.

Le immagini estratte dallo ZIP vengono passate allo `StorageService`: se Azure è configurato, il file viene caricato direttamente nel Blob Storage cloud; altrimenti viene salvato localmente in modalità Dual-Mode.

# Capitolo 7

## Frontend — Dashboard Istituzionale React

### 7.1 Architettura a Servizi e Viste

Il frontend è strutturato seguendo il principio della **Separation of Concerns** in livelli distinti:

- **Componenti di presentazione** (`src/app/components/`): elementi UI riutilizzabili (`DocCard`, `DocFormModal`, `ChatModal`, ecc.).
- **Servizi** (`src/app/services/`): incapsulano le chiamate API REST e la logica di business, mantenendo le views snelle.
- **Viste** (`src/app/views/`): orchestrano la composizione e il routing, tenendo in stati locali i dati di pagina.

#### 7.1.1 Componenti e Viste principali

| File                               | Responsabilità                                                  |
|------------------------------------|-----------------------------------------------------------------|
| <code>HomeView.tsx</code>          | Home pubblica: ricerca, filtri, risultati, upload OCR           |
| <code>AdminView.tsx</code>         | Pannello operatore: CRUD documenti, bulk import, configurazione |
| <code>DetailView.tsx</code>        | Dettaglio atto: testo OCR, metadati, modifica, chatbot          |
| <code>ResetPasswordView.tsx</code> | Pagina reset password con token da email                        |
| <code>DocCard.tsx</code>           | Scheda documento con badge affinità AI                          |
| <code>ChatModal.tsx</code>         | Chatbot RAG con storico conversazione                           |
| <code>DocFormModal.tsx</code>      | Form inserimento / modifica con pre-compilazione OCR            |

#### 7.1.2 Servizi Frontend

| Servizio                         | Responsabilità                                              |
|----------------------------------|-------------------------------------------------------------|
| <code>authService.ts</code>      | Token JWT, login, logout, gestione utenti, trigger cron     |
| <code>documentiService.ts</code> | CRUD documenti, export ZIP, eliminazione bulk               |
| <code>ricercaService.ts</code>   | Ricerca testuale (con filtri) e ricerca per immagine        |
| <code>entitaService.ts</code>    | CRUD entità di configurazione (uffici, tipologie, frazioni) |
| <code>chatService.ts</code>      | Chiamate al chatbot RAG                                     |
| <code>auditService.ts</code>     | Lettura del Registro Attività                               |
| <code>api.ts</code>              | Interceptor HTTP: token JWT, gestione errori 401            |

## 7.2 Design System

- **Framework UI:** Componenti base accessibili (Radix UI) stilizzati tramite **Tailwind CSS** (stile *shadcn/ui*), garantendo conformità WCAG.
- **Palette e Temi:** Sistema cromatico flessibile basato su variabili CSS.
- **Tipografia e Iconografia:** Font di sistema sans-serif ottimizzati per la leggibilità e set di icone coerente basato su **Lucide React**.
- **Responsive Design:** Layout fluido e componenti adattivi (es. `react-resizable-panels`) progettati *mobile-first*.

## 7.3 Funzionalità UX Avanzate

### 7.3.1 Filtri Multi-Selezione con Logica AND/OR

I cinque menu filtro della home page (Tipologia, Anno, Ufficio, Firmatario, Frazione) supportano la **selezione multipla** tramite checkbox. Uno switch **TUTTI/ALMENO UNO** per ogni dimensione configura la logica booleana. Ogni menu include una barra di ricerca interna per filtrare le opzioni in tempo reale, utile con liste lunghe di firmatari o frazioni. I valori vengono serializzati come array di query string e inviati all'endpoint `/search`, che li elabora sia in AND che in OR lato backend.

### 7.3.2 Indicatore di Affinità Semantica

Ogni DocCard mostra in overlay sulla miniatura un **badge percentuale** (cosine similarity  $\times 100$ ) restituito dal backend. La sezione spiegazione AI ha altezza fissa di 140px, garantendo allineamento uniforme tra card della stessa riga. Il prompt inviato ad Azure OpenAI richiede una sintesi istituzionale di **massimo 2-3 frasi**

per le spiegazioni dei documenti correlati, al fine di mantenere le card equilibrate e leggibili senza imporre limiti di parole troppo rigidi.

### 7.3.3 Selezione Multipla e Bulk Export

Cliccando su uno o più documenti appare un menu fluttuante in basso con le azioni **Esporta ZIP** ed **Elimina**, permettendo operazioni in massa senza aprire ogni scheda individualmente.

### 7.3.4 Sicurezza Token: `sessionStorage` vs `localStorage`

Il token JWT dell'amministratore viene conservato in `sessionStorage` anziché in `localStorage`. A differenza del `localStorage` (persistente anche dopo la chiusura del browser), il `sessionStorage` viene azzerato alla chiusura della tab. Questo impedisce il ricaricamento automatico del pannello admin se l'applicazione viene riaperta, garantendo che ogni sessione parta sempre dallo stato "non autenticato".

## 7.4 URL Relativi e Proxy

Tutte le chiamate API usano URL relativi (`/api/v1/...`), rendendo il frontend indipendente dall'indirizzo esatto del backend. La configurazione dettagliata del proxy Nginx e il meccanismo `envsubst` per l'iniezione dinamica di `BACKEND_URL` sono descritti nel Capitolo 8.

## 7.5 Autenticazione Stateless nel Frontend

Il Bearer Token JWT viene allegato automaticamente a ogni richiesta HTTP protetta tramite la funzione `apiFetch` nel file `api.ts`. Il gestore globale degli errori 401 provoca un logout forzato (reload della pagina), con un'eccezione esplicita per l'endpoint `/auth/login`, che può legittimamente rispondere con 401 senza provocarne il ricaricamento (la schermata di login mostra invece il messaggio di errore).

# Capitolo 8

## Containerizzazione e Deploy Cloud (Azure)

L'intera applicazione è containerizzata tramite **Docker** e deployata su **Azure Container Apps**: ogni servizio (backend Python e frontend React) è incapsulato in un'immagine dedicata, costruita con strategie ottimizzate per la produzione.

### 8.1 Dockerfile Backend

Il backend parte dall'immagine base `python:3.9-slim`, che esclude i pacchetti non necessari riducendo la superficie d'attacco. Le fasi principali sono:

1. **Dipendenze di sistema:** si installano solo `build-essential` e `curl` (le librerie minime per compilare alcuni pacchetti Python).
2. **Requisiti Python:** la copia separata di `requirements.txt` prima del codice sorgente sfrutta la cache a layer di Docker: se le dipendenze non cambiano, questo step non viene rieseguito.
3. **Pre-download modelli AI al build time:** il modello linguistico `it_core_news_sm` (spaCy) e il modello di embedding `paraphrase-multilingual-MiniLM-L12-v2` (SentenceTransformer) vengono scaricati durante la build, non al primo avvio. Questo elimina la latenza iniziale in produzione (fino a 90 secondi) e rende l'immagine autonoma.
4. **Avvio:** il server `uvicorn` viene lanciato in ascolto su `0.0.0.0:8000` per accettare connessioni dall'esterno del container.

#### Il ruolo del healthcheck Docker

SentenceTransformer richiede 30–90 secondi al primo avvio per caricare i pesi in memoria. Il `start_period: 120s` del healthcheck garantisce che il container venga marcato come *healthy* solo dopo che la pipeline NLP è completamente operativa, evitando errori sulle prime richieste.



## 8.2 Multi-Stage Build del Frontend

Il Dockerfile del frontend è strutturato in **due stage** distinti per ridurre al minimo la dimensione finale dell'immagine:

**Stage 1 --- build (Node.js 22 Alpine):** Si installa **pnpm** tramite **corepack**, si copiano le dipendenze e si esegue **pnpm run build**. Il risultato è la cartella **/dist** con l'applicazione React compilata. Questo stage produce un'immagine temporanea da ~1.5 GB che non viene mai pubblicata.

**Stage 2 --- runtime (Nginx Alpine):** Si parte da un'immagine Nginx leggera (~25 MB) e si copiano **esclusivamente** i file statici generati dal Stage 1 con la direttiva **COPY --from=build**. Il risultato è un'immagine di produzione di soli ~25 MB, senza dipendenze di sviluppo, compilatori o codice sorgente.

## 8.3 Nginx con envsubst

Il container frontend usa **Nginx Alpine** come web server. Il problema classico del frontend containerizzato è che l'URL del backend non è noto al momento della *build*: cambia tra sviluppo locale, staging e produzione Azure.

La soluzione adottata è il meccanismo **envsubst** (dalla libreria **gettext**):

1. Il file **nginx.conf** usa il **segnaposto** **\${BACKEND\_URL}** invece di un indirizzo hardcoded.
2. Al momento dell'avvio del container, il **CMD** del Dockerfile esegue **envsubst** che sostituisce **\${BACKEND\_URL}** con il valore reale della variabile d'ambiente, generando il file di configurazione definitivo.
3. Nginx viene poi avviato con il file già compilato.

Il template **nginx.conf** definisce tre **location** block distinti:

**/api/:** tutte le richieste API vengono inoltrate al backend FastAPI tramite **proxy\_pass** con timeout esteso a 120s (per le operazioni OCR che possono durare fino a 30-90 secondi).

**/static/:** i file statici (immagini in modalità locale) vengono proxati al backend; in produzione Azure vengono serviti direttamente tramite URL del Blob Storage.

**/:** tutti gli altri percorsi servono **index.html** con **try\_files \$uri \$uri/ /index.html**, abilitando il routing lato client di React Router senza 404 su navigazione diretta.

### Valore di default per sviluppo locale

Il **CMD** del Dockerfile usa la sintassi **\${BACKEND\_URL:-http://backend:8000}**: se la variabile non è impostata, il fallback è **http://backend:8000**, che corrisponde al nome del servizio backend in Docker Compose. In produzione Azure, **BACKEND\_URL** viene impostato dall'esterno nel pannello *Environment Variables*

del Container App.

## 8.4 Deploy su Azure Container Apps

Il deploy avviene in quattro passi principali:

1. **Provisioning delle risorse Azure:** creazione dell'Azure Container Registry (`acrdocumatch`), del database PostgreSQL Flexible Server con il DB `documatch_db`, dell'Azure Blob Storage per le immagini, del servizio Azure AI Document Intelligence (OCR) e dell'Azure OpenAI Service con il deployment del modello GPT 5.4 Mini.
2. **Build e Push:** le immagini Docker vengono costruite localmente (o tramite pipeline CI/CD) e caricate nell'Azure Container Registry con `az acr build` o `docker push`.
3. **Creazione dei Container Apps:** il backend viene deployato su porta 8000 con *Ingress External* e le variabili d'ambiente: `DATABASE_URL`, `SECRET_KEY`, `AZURE_DOCUMENT_INTELLIGENCE_KEY/ENDPOINT`, `AZURE_STORAGE_CONNECTION_STRING`, `AZURE_OPENAI_KEY/ENDPOINT`.  
Il frontend viene deployato su porta 80 con `BACKEND_URL` impostato all'URL del Container App del backend.
4. **Configurazione CORS:** la variabile `CORS_ALLOWED_ORIGINS` viene impostata all'URL esatto del frontend. Con `ENVIRONMENT=production` e CORS wildcard (\*), il backend si rifiuta di avviarsi per prevenire accessi non autorizzati da origini arbitrarie.

# Capitolo 9

## Sicurezza e Autenticazione

### 9.1 Token JWT (HS256 / HMAC-SHA256) Stateless

Autenticazione stateless senza sessioni server-side. Viene usata la libreria **PyJWT** con algoritmo **HS256** per generare token JWT standard (Header.Payload.Signature) firmati con la **SECRET\_KEY**:

### 9.2 RBAC: API Pubbliche e Protette in Scrittura

Il framework implementa un sistema di controllo degli accessi (Role-Based Access Control) granulare a livello di endpoint:

- **Lettura pubblica:** Trattandosi di un simulatore di albo pretorio, le operazioni di lettura (GET) e di ricerca semantica (POST per image-search) non richiedono alcun token JWT, rendendo i documenti finali accessibili a tutti i cittadini senza autenticazione.
- **Scrittura protetta:** Qualsiasi endpoint che implichi una modifica dei dati o dei modelli (creazione, aggiornamento, eliminazione documenti, generazione OCR e ricalcolo embeddings) è rigidamente protetto lato server dalla dipendenza `Depends(get_current_admin)`. In assenza di un header **Authorization: Bearer <token>** valido, FastAPI respinge intrinsecamente la chiamata con errore HTTP 401 Unauthorized, rendendo impossibile l'alterazione del database eludendo la UI del frontend.

### 9.3 Password Hashing con Bcrypt

Bcrypt implementa *key stretching*: ogni hash richiede un tempo di calcolo parametrizzabile, rendendo il brute-force da GPU impraticabile.

### 9.4 Reset Password con Token Monouso

1. L'operatore richiede il reset con il proprio username.

2. Il backend genera un token casuale (128 bit, hex) con scadenza 15 minuti.
3. Il token viene salvato nel DB. Un link viene inviato via email.
4. L'operatore clicca il link; il backend verifica il token e aggiorna la password (bcrypt). Il token viene invalidato impostandolo a `null`.

## 9.5 Protezione Anti-ZipBomb

L'endpoint di caricamento massivo di documenti tramite archivi ZIP (`/api/v1/documents/import-bulk`) è vulnerabile ad attacchi di tipo *Zip Bomb* o *Decompression Bomb*, dove un piccolo archivio (es. 42 KB) si decompime in Petabyte di dati riempiendo la memoria del server. Per mitigare questa vulnerabilità, il codice ispeziona i metadati dell'archivio (tramite `z.infolist()`) prima di effettuare la lettura. Vengono imposti i seguenti limiti rigidi:

- `MAX_ZIP_FILES`: massimo 1000 file nell'archivio.
- `MAX_UNCOMPRESSED_SIZE`: dimensione massima espansa bloccata a 100 MB.

Se questi limiti vengono superati, l'estrazione è annullata e la richiesta respinta con errore 400 (Bad Request).

## 9.6 Sistema di Accountability (Audit Log)

Un sistema documentale richiede la tracciabilità di tutte le azioni sensibili per la sicurezza aziendale. È stato implementato un `AuditLog` su database relazionale:

- Il modello traccia `timestamp`, `username`, `action` (es. `CREATE_DOC`, `UPDATE_DOC`, `MASS_IMPORT`) e dettagli esplicativi.
- Ogni endpoint REST che esegue modifiche salva a posteriori il log dell'operazione.
- Nell'interfaccia React è stata aggiunta un'area amministrativa "Registro Attività" visibile solo agli amministratori autorizzati (Supreme Admin) per un'analisi real-time delle scritture.

# Capitolo 10

## Sistema di Notifiche Schedulate

### 10.1 APScheduler in Background

Lo scheduler viene avviato nel lifecycle `lifespan` di FastAPI tramite **APScheduler** con un cron job configurato per le ore **08:00 ogni giorno**. La logica di notifica implementa un sistema di **escalation graduale**:

- **15 giorni prima:** prima notifica email a tutti gli operatori registrati.
- **7 giorni prima:** secondo promemoria.
- **1 giorno prima:** notifica urgente di imminente scadenza.

Per evitare spam, il sistema controlla che la notifica per un determinato documento non sia già stata inviata nella stessa finestra temporale.

L'endpoint `POST /api/v1/documents/trigger-expiry-check` consente all'amministratore di forzare il controllo manualmente dalla UI (pulsante "Trigger Email Scadenze" nella scheda Impostazioni del pannello admin), utile per test e demo. In modalità demo, il trigger considera qualsiasi documento in scadenza nei prossimi 15 giorni (non solo quelli esattamente ai tre giorni soglia), inviando immediatamente le notifiche per tutti i documenti in procinto di scadere.

# Capitolo 11

## Integrazione dell'IA Generativa — Dichiarazione di Trasparenza

### 11.1 Strumento Utilizzato

In conformità con le indicazioni del docente, si dichiara che per lo sviluppo del progetto è stato utilizzato lo strumento di AI Generativa **Antigravity IDE** (basato su Google Gemini e Anthropic Claude Sonnet), integrato in Visual Studio Code.

L'IA è stata usata come **strumento di accelerazione**, non come sostituto della progettazione. Ogni componente è stato compreso, rivisto e adattato.

### 11.2 Distribuzione del Lavoro

| Area                   | Contributo AI                       | Contributo Studente                 |
|------------------------|-------------------------------------|-------------------------------------|
| <b>Architettura</b>    | Pattern Dual-Mode, SO-LID/DIP       | Scelte Azure, adattamento dominio   |
| <b>Pipeline NLP</b>    | Bozza spaCy + Sentence-Transformers | Tuning, integrazione, hybrid boost  |
| <b>Frontend React</b>  | Bozza componenti e hook TypeScript  | UX istituzionale, CSS custom        |
| <b>Backend FastAPI</b> | Struttura router, DI                | Logica business, filtri, migrazioni |
| <b>Scheduler</b>       | Configurazione APScheduler          | Dual-mode demo/cron, lifespan       |
| <b>Deploy Azure</b>    | Pattern envsubst Nginx              | Test, verifica, adattamento         |
| <b>Sicurezza</b>       | HMAC token, bcrypt                  | Reset flow, CORS produzione         |
| <b>Relazione</b>       | Struttura, formattazione            | Contenuto tecnico, revisione        |

Tabella 11.1: Distribuzione lavoro studente / AI

## 11.3 Prompt Principali Utilizzati

Di seguito vengono riportati i prompt reali estratti dalle conversazioni esportate con l'Intelligenza Artificiale, a dimostrazione dell'esteso e complesso flusso di lavoro svolto per la realizzazione di DocuMatch.

### 11.3.1 Fase 1: UI/UX e Frontend React

#### Prompt 1 — Progettazione Interfaccia Base

**Utente:** *‘Crea un sito web per la gestione delle schede documentali del comune di Montalto Uffugo. La piattaforma si chiamerà DocuMatch e deve avere una home page con tutte le schede, una barra di ricerca con filtri combinabili e una modalità di ricerca OCR da immagine tramite uno switch dedicato. L’area admin deve permettere la modifica delle schede.’*

**Risposta e Azioni:** L’AI ha generato l’architettura React in Documatch/src/app, implementando il layout grid a 3 colonne, la barra di ricerca full-text e i filtri combinabili.

#### Prompt 2 — Ricerca OCR per Immagine

**Utente:** *‘Aggiungi una sezione di ricerca tramite OCR con un’area di caricamento drag-and-drop ben visibile. Sostituisci lo switch poco evidente con due tab distinte in cima alla barra di ricerca: una per la ricerca testuale e una per la ricerca per immagine.’*

**Risposta e Azioni:** L’AI ha riprogettato la barra di ricerca aggiungendo due tab visibili in cima (Ricerca testuale / Ricerca per immagine). È stata implementata una dropzone drag-and-drop per i formati JPG/PNG/PDF.

#### Prompt 3 — Integrazione OCR nel Form Admin

**Utente:** *‘Aggiungi l’upload OCR simulato direttamente nel form di inserimento.’*

**Risposta e Azioni:** Modificato il DocFormModal.tsx. L’AI ha aggiunto una sezione di caricamento file in cima al form che, al drop dell’immagine, avvia un’analisi OCR simulata e pre-compila automaticamente i campi Numero, Nome, Descrizione e Tipologia.

#### Prompt 4 — Configurazione Dinamica Metadati

**Utente:** *‘Trasforma i campi a testo libero (Uffici, Firmatari) in valori preimpostati e modificabili dall’amministratore. Aggiungi una sezione “Configurazione” nell’area admin che permetta di gestire queste entità tramite interfaccia grafica anziché inserimento manuale.’*

**Risposta e Azioni:** Aggiunta la sezione “Configurazione” nell’area admin. L’AI ha trasformato i campi di testo libero (Uffici, Firmatari) in griglie di pill/checkbox multi-select estrapolate dal database, migliorando la consistenza

dei dati.

### 11.3.2 Fase 2: Backend e Integrazione API

#### Prompt 5 — Allineamento Tipi Backend/Frontend

**Utente:** *‘Verifica che il backend sia correttamente allineato con il frontend di Documatch. Controlla che gli schemi Pydantic e le interfacce TypeScript siano consistenti per tutti i campi, le date e le entità collegate, e correggi ogni discrepanza trovata.’*

**Risposta e Azioni:** L’AI ha scansionato `Documatch/src/app/types` e `backend/app/schemas/`. Notando discrepanze sui formati di date e sulle entità collegate, ha unificato le dichiarazioni TypeScript e Pydantic.

#### Prompt 6 — Semplificazione Schemi e Documentazione Codice

**Utente:** *‘Semplifica gli schemi Pydantic dove possibile ed aggiungi commenti esplicativi diffusi in tutto il codice. Il sistema deve essere comprensibile.’*

**Risposta e Azioni:** L’AI ha analizzato l’albero dei modelli. Anzichè rimuovere funzionalità, ha unificato e ripulito ulteriormente gli schemi Pydantic e le dichiarazioni, commentando ampiamente il codice.

### 11.3.3 Fase 3: Intelligenza Artificiale e Cloud

#### Prompt 7 — Porting su Cloud Azure

**Utente:** *‘Implementa il porting completo del progetto su Microsoft Azure. Adatta il backend per usare PostgreSQL tramite variabile d’ambiente, sostituisci i mock locali con gli SDK Azure reali per Blob Storage e Document Intelligence, e configura i container per il deploy su Azure Container Apps.’*

**Risposta e Azioni:** L’AI ha tracciato la roadmap infrastrutturale: adattamento di `database.py` per PostgreSQL via URL d’ambiente, sostituzione dei Mock locali con SDK Azure reali (Blob Storage) e configurazione dei container. Le modifiche sono state implementate in `backend/app/core/`.

#### Prompt 8 — Chatbot RAG per interrogazione Atti

**Utente:** *‘Implementa un chatbot interattivo stile AI generativa che permetta all’utente di interrogare direttamente i documenti dell’archivio. Usa Azure OpenAI con pattern RAG (Retrieval-Augmented Generation): cerca i documenti più rilevanti con la ricerca vettoriale e inietta nel contesto del prompt per ottenere risposte basate esclusivamente sull’archivio locale.’*

**Risposta e Azioni:** L’AI ha esteso `generative_ai.py` e integrato l’API di Azure OpenAI per implementare un Chatbot Documentale RAG che risponde in modo semantico basandosi esclusivamente sul contenuto del documento



corrente o sui metadati estratti.

#### Prompt 9 — Gestione Campi Vuoti nell'Estrazione OCR

**Utente:** *‘Gestisci correttamente i campi non trovati dall’OCR: i campi stringa non rilevati devono essere impostati a `null`, le liste a array vuoto. Non inserire mai etichette di default come “Altro” o “Sconosciuto” quando l’informazione non è presente nel testo.’*

**Risposta e Azioni:** L’AI ha adeguato i servizi di validazione in `nlp.py` e nel prompt di Azure OpenAI, assicurandosi che i campi non estratti venissero impostati correttamente a `null` o lista vuota, senza inserire etichette errate.

### 11.3.4 Fase 4: DevOps, Debugging e Consegna

#### Prompt 10 — Correzione Healthcheck e Avvio Docker

**Utente:** *‘Risolvi l’errore di avvio del container backend (`dependency failed to start`). Il backend impiega troppo ad avviarsi a causa del caricamento dei modelli AI: aumenta il `start_period` dell’healthcheck in `docker-compose.yml` e correggi le policy `depends_on` affinché il frontend attenda la piena operatività del backend prima di ricevere traffico.’*

**Risposta e Azioni:** Verificando i log, l’AI ha individuato che il caricamento dei modelli AI locali causava il timeout dell’healthcheck di Docker. Ha corretto `docker-compose.yml` aumentando gli `start_period` e sistemato le policy di `depend_on` tra frontend e backend.

#### Prompt 11 — Code Audit e Pulizia della Codebase

**Utente:** *‘Fai un controllo sul progetto. Controlla bene se c’è del codice morto e rimuovilo, controlla la pulizia generale.’*

**Risposta e Azioni:** L’AI ha eseguito uno scan completo. Sono stati rimossi script di test obsoleti, ripuliti gli import dismessi in `main.py` e `App.tsx`, e rimossi mockup temporanei superati dall’effettiva implementazione API, ottenendo una repository pulita per la consegna.

### 11.3.5 Fase 5: UX Avanzata e Correzioni Frontend

#### Prompt 12 — Correzione Credenziali Errate Ricaricano la Pagina

**Utente:** *‘Ogni volta che sbaglio le credenziali la pagina si aggiorna e mi riporta alla home.’*

**Risposta e Azioni:** L’AI ha individuato il bug in `api.ts`: il gestore globale degli errori 401 ricaricava la pagina anche durante il login, prima che il messaggio di errore potesse essere mostrato. È stata aggiunta un’eccezione

esplicita per l'endpoint `/auth/login`, consentendo al componente di catturare l'errore e mostrare la scritta rossa senza redirect.

### Prompt 13 — Rimozione Completa del Campo Numero Protocollo

**Utente:** *‘Rimuovi completamente il campo numero dall'intero sistema.’*

**Risposta e Azioni:** L'AI ha rimosso la colonna `numero` dal modello `SQ-LAlchemy`, dagli schemi `Pydantic`, dal form React (`DocFormModal.tsx`), dalle `DocCard`, dalla tabella `admin` e dai dati mock dell'OCR, mantenendo la coerenza full-stack senza lasciare dati orfani nel database.

### Prompt 14 — Stabilizzazione Visiva della Barra di Scorrimento

**Utente:** *‘Si ha un effetto visivo fastidioso quando appare o scompare la barra laterale di scorrimento.’*

**Risposta e Azioni:** Quando il numero di documenti filtrati non generava overflow, il layout si “scattava” lateralmente. L'AI ha aggiunto la regola CSS `overflow-y: scroll` sul tag `html` per riservare permanentemente lo spazio della scrollbar, eliminando ogni spostamento visivo al cambio dei risultati.

### Prompt 15 — Riorganizzazione Area Admin con DocCard Integrate

**Utente:** *‘Nell'area admin mostra solo la possibilità di aggiunta. La modifica e l'eliminazione devono potersi fare direttamente dalla scheda, non dalla lista.’*

**Risposta e Azioni:** L'AI ha rimosso la tabella-lista da `AdminView.tsx`, trasformandola in un pannello compatto con pulsanti (Aggiungi / Importa Massivo / Configurazione). I bottoni Modifica ed Elimina sono stati inseriti direttamente in `DocCard.tsx` e `DetailView.tsx`, visibili solo quando l'utente è autenticato come amministratore.

### Prompt 16 — Barra di Ricerca nel Form e Selezione Multipla Frazioni

**Utente:** *‘Nel form di inserimento aggiungi la ricerca per filtrare le opzioni, e la possibilità di scegliere più frazioni.’*

**Risposta e Azioni:** L'AI ha esteso il modello `Document` trasformando il campo `frazione` (stringa singola) in un array JSON serializzato, creato una migrazione additive per `SQLite` e aggiornato `DocFormModal.tsx` con un campo di ricerca che filtra in tempo reale le pill per Tipologia, Uffici, Firmatari e Frazioni.

### Prompt 17 — Logica Wildcard “Tutte le Frazioni”

**Utente:** *‘Implementa la logica per cui se viene selezionata una frazione devono essere considerati anche i documenti validi per tutte le frazioni.’*

**Risposta e Azioni:** L'AI ha aggiornato l'endpoint `/search` in `documents.py`: quando viene applicato il filtro su una frazione specifica, vengono ora inclusi automaticamente anche i documenti che hanno `frazioni` impostato

a "TUTTE", trattandolo come wildcard istituzionale a copertura dell'intero territorio comunale.

### 11.3.6 Fase 6: Deploy Cloud e Ottimizzazioni Azure

#### Prompt 18 — Storage Blob per Importazione Massiva

**Utente:** *‘Nell’importazione massiva le immagini non vengono salvate su Azure Blob ma solo in locale.’*

**Risposta e Azioni:** L’AI ha riscritto il gestore ZIP in `documents.py`: ogni immagine estratta dall’archivio viene ora passata al `StorageService`, che la carica su Azure Blob Storage (o la salva localmente in Dual-Mode). È stata aggiornata anche l’esportazione ZIP per recuperare le immagini correttamente sia dal blob cloud che dal filesystem locale.

#### Prompt 19 — Chatbot con Memoria, Contesto e Senza Markdown

**Utente:** *‘Il chatbot risponde male, inserisce asterischi, non ricorda la conversazione e confonde il documento corrente con gli altri. Specifica nei prompt di non utilizzare markdown e il documento corrente deve essere sempre ben distinto dagli altri.’*

**Risposta e Azioni:** L’AI ha rielaborato il prompt di sistema in `chat.py` con direttive rigide (niente Markdown, nessun asterisco) e ha distinto il documento corrente dagli altri usando l’etichetta `[DOCUMENTO CORRENTE ATTUALMENTE APERTO]`. Il frontend è stato corretto per inviare lo storico completo della conversazione ad ogni chiamata, abilitando la memoria della chat.

#### Prompt 20 — Correzione Estrazione Titolo OCR tramite LLM

**Utente:** *‘Il titolo estratto tramite OCR è sbagliato: scrive la descrizione invece del numero e tipo dell’atto.’*

**Risposta e Azioni:** Il cambio di modello OpenAI aveva alterato l’interpretazione del campo `nome`. L’AI ha reso il prompt più prescrittivo: *“Includi SEMPRE il tipo e il numero dell’atto se presenti (es. ‘Ordinanza Sindacale n. 12’). Evita descrizioni generiche.”*, separando nettamente il titolo identificativo dalla descrizione testuale.

#### Prompt 21 — Risoluzione Bug Email Notifiche di Scadenza

**Utente:** *‘Il sistema di email per le notifiche di scadenza non funziona.’*

**Risposta e Azioni:** Il pulsante “Trigger Email Scadenze” usava una funzione `apiFetch` non importata nel componente React. L’AI ha spostato la chiamata nel corretto `authService.ts` e ha migliorato la logica backend: il cron job invia avvisi a 15, 7 e 1 giorno dalla scadenza (con gradazione di urgenza), mentre la modalità demo include tutti gli atti in scadenza entro 15 giorni.

#### Prompt 22 — Visibilità Password e Ancoraggio Spiegazione AI

**Utente:** *‘Aggiungi un'icona occhio in tutti i campi password, e ancora la descrizione di similarità in basso nelle card.’*

**Risposta e Azioni:** L'AI ha aggiunto il toggle di visibilità password (icona Eye/EyeOff da lucide-react) nei form di login e reset. In DocCard.tsx ha usato il layout Flexbox con `h-full` e `flex-1` per spingere il pannello della spiegazione AI sempre al fondo della card, garantendo allineamento uniforme in griglia multi-colonna.

#### Prompt 23 — Allineamento Tipologie Esistenti nel Database

**Utente:** *‘Allinea nel database le tipologie dei documenti già esistenti per renderle consistenti con quelle configurate nel sistema.’*

**Risposta e Azioni:** L'AI ha eseguito uno script di migrazione dati che ha aggiornato 15 record con le vecchie diciture abbreviate (“Determina”, “Ordinanza”, “Atto”) convertendole nelle categorie estese ufficiali (“Determinazione Dirigenziale”, “Ordinanza Sindacale”, “Altro”), garantendo la consistenza con i filtri dell'interfaccia.

#### Prompt 24 — Protezione Anti-Zip Bomb per Importazione Massiva

**Utente:** *‘Aggiungi una protezione contro possibili Zip Bomb nell'endpoint di importazione massiva.’*

**Risposta e Azioni:** L'AI ha aggiunto in documents.py un controllo preventivo sui metadati dell'archivio ZIP prima dell'estrazione: vengono imposti il limite di 1000 file e 100 MB di dimensione decompressa massima. Se superati, l'operazione viene annullata con errore HTTP 400, prevenendo attacchi di tipo Decompression Bomb.

# Capitolo 12

## Conclusioni e Sviluppi Futuri

### 12.1 Risultati Raggiunti

Il progetto **DocuMatch** ha centrato pienamente tutti gli obiettivi accademici e tecnologici della traccia di Sistemi Distribuiti e Cloud Computing.

L'adozione di **Azure Container Apps** al posto delle classiche VM IaaS ha dimostrato concretamente i vantaggi del paradigma PaaS: auto-scaling, networking e TLS automatici, zero gestione del sistema operativo.

La **pipeline AI** (spaCy NER, SentenceTransformers, Azure OpenAI, Azure AI Document Intelligence) ha trasformato un semplice archivio digitale in uno strumento documentale intelligente, con una ricerca semantica che supera ampiamente i sistemi keyword-based tradizionali.

Il pattern **Dual-Mode** ha consentito uno sviluppo efficiente senza sprecare crediti Azure, garantendo al contempo la piena funzionalità in produzione cloud.

### 12.2 Competenze Acquisite

- Progettazione di architetture cloud-native (IaaS, PaaS, SaaS).
- Containerizzazione Docker e ottimizzazione con Multi-Stage Build.
- Utilizzo degli SDK Azure (Blob Storage, Document Intelligence, OpenAI).
- Implementazione di algoritmi di similarità semantica su embedding vettoriali.
- Sviluppo di API REST ASGI con FastAPI e SQLAlchemy ORM.
- Utilizzo documentato e critico dell'IA Generativa nello sviluppo software.

### 12.3 Sviluppi Futuri

- **Supporto PDF Nativo:** integrare PyPDF2 o pdfminer per PDF multipagina.
- **Ricerca con pgvector:** sostituire il calcolo in-memory con l'estensione PostgreSQL pgvector per ricerca ANN a livello database.

- **Autenticazione SPID / CIE:** identità digitale federata ministeriale.
- **Fine-Tuning dei Modelli:** corpus di atti burocratici italiani per massimizzare la precisione nel gergo tecnico-legale.
- **Indicizzazione Incrementale:** aggiornare solo gli embedding modificati.